StudyFix

Tài liệu học và chuẩn bị vấn đáp

Kiến trúc hệ thống • TCP Socket • Giao thức riêng • Luồng xử lý • Cơ sở dữ liệu • Chấm bài

Dành cho buổi báo cáo trên lớp ngày 4 tháng 11 năm 2026

Điều cần hiểu trước tiên

StudyFix là ứng dụng học và thực hành Java theo nhóm. Giáo viên tổ chức phòng học, giảng trên code chung, tách nhánh cho sinh viên và chọn lời giải làm code chung mới. Hệ thống còn hỗ trợ chat, luyện tập, giao bài, cuộc thi và chia sẻ màn hình.

Kiến trúc chính là client–server. Server Java giữ quyền quyết định về danh tính, phòng, code và kết quả chấm. Dữ liệu lưu trong SQLite. Có hai loại client: trình duyệt web và ứng dụng Java Swing. Hai client dùng các đường kết nối khác nhau nên cần trình bày đúng khi được hỏi.

Cách học tài liệu trong một buổi tối

Lượt 1 khoảng 30 phút: đọc mục 1–6 để hiểu kiến trúc và TCP. Lượt 2 khoảng 45 phút: đọc mục 7–15, tự kể lại luồng đăng nhập, sửa code và chấm bài. Lượt 3 khoảng 30 phút: tập nói theo mục 18 và trả lời mục 19–22. Trước giờ báo cáo, ôn mười ý ở mục 22 và chuẩn bị demo theo mục 17. Trong Word có thể bật View rồi Navigation Pane để tìm theo tiêu đề.

Ba câu có thể dùng để mở đầu

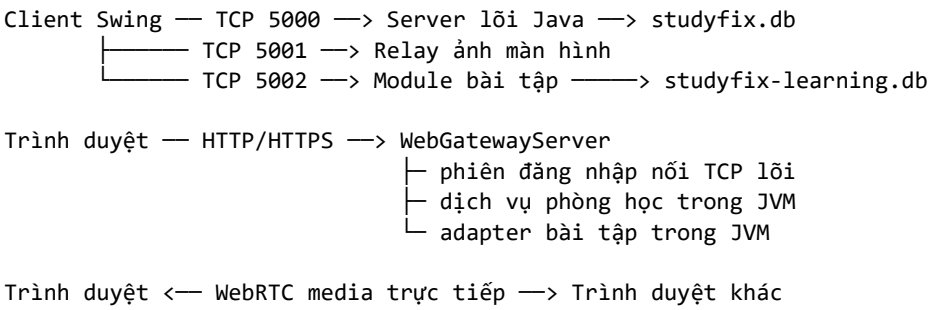
“StudyFix của nhóm em hỗ trợ học Java cộng tác. Hệ thống dùng server Java và SQLite, có giao diện web cùng client Swing. Nhóm thiết kế giao thức tầng ứng dụng riêng trên TCP cho các kênh của client Java, còn web dùng HTTP/HTTPS và WebRTC cho media.”

Phạm vi học

Tài liệu đi từ khái niệm dễ hiểu đến cách mã nguồn triển khai, rồi đến câu trả lời vấn đáp. Khi nêu một giới hạn hay công nghệ, hãy gắn nó với đúng thành phần; không suy rộng rằng mọi chức năng đều dùng TCP Socket trực tiếp.

1 Bức tranh kiến trúc tổng thể

Có thể hình dung server là trung tâm điều phối lớp học: người dùng gửi yêu cầu, server kiểm tra quyền và dữ liệu rồi trả kết quả. Giao diện chỉ là nơi nhập và hiển thị; người dùng sửa giao diện cũng không được tự cấp quyền ở server.



Ba nhóm trách nhiệm

Tầng giao diện gồm Swing hoặc HTML/CSS/JavaScript. Tầng xử lý gồm kết nối, kiểm tra quyền, phòng học, cộng tác code, giao bài và chấm. Tầng lưu trữ dùng JDBC và SQLite. Đây là cách mô tả phân lớp theo trách nhiệm; không có nghĩa dự án chạy ba server độc lập.

Cách triển khai thực tế

Launcher đầy đủ khởi động server lõi, relay màn hình, module bài tập và web gateway trong cùng một JVM. Vì vậy nên gọi đây là ứng dụng server được chia thành các module. Việc có nhiều cổng chưa đủ để gọi hệ thống là microservices.

Điểm cần nói chính xác

Server tập trung quản lý nghiệp vụ, nhưng media web đi qua kết nối WebRTC giữa các trình duyệt. Gateway truyền tín hiệu thiết lập cuộc gọi, không đồng nghĩa toàn bộ video phải đi qua server. Client Swing chia sẻ màn hình theo đường relay TCP khác với web.

Đọc mã nguồn: server/src/studyfix/assignment/AssignmentEnabledServerMain.java; Server.java; WebGatewayServer.java; README mục 3 và 6.

2 Cấu trúc dự án và bản đồ mã nguồn

Vị trí	Chứa gì	Khi thầy hỏi
server/src/studyfix	Server Java, phòng, giao thức, database	Nghiệp vụ xử lý ở đâu
server/src/studyfix/assignment	Đề, bài nộp, cuộc thi, chấm Docker	Chấm bài như thế nào
client/src/main/java	Giao diện Swing và TCP client	Client mở socket ở đâu
web-client/static	HTML, CSS, JS dạng ES Modules	Giao diện web kết nối ra sao
server/test và web-client/tests	Kiểm thử Java và JavaScript	Đã có cách kiểm tra gì
server/data	Hai database SQLite lúc chạy	Restart có mất dữ liệu không
build, dist, target	Kết quả biên dịch và đóng gói	Mã nguồn khác file class ra sao

Các lớp server cần nhớ

Server nhận kết nối và quản lý các phòng/client. ClientHandler đọc lệnh của một kết nối. Protocol đóng và đọc frame. Room quản lý thành viên, code, lượt sửa và phiên bản. Store làm việc với dữ liệu lõi. ClassroomService điều phối Master và nhánh; ClassroomRepository lưu dữ liệu phòng học.

Các lớp client cần nhớ

TcpControlConnection giữ kết nối TCP điều khiển. FrameCodec đọc/ghi frame. Các service tạo lệnh theo chức năng. RoomController nhận thông báo và cập nhật trạng thái hiển thị. Các lớp ui hiển thị cửa sổ. AssignmentConnection dùng kết nối bài tập riêng.

Cấu hình và công cụ build

Server dùng Ant và cấu hình hiện tại yêu cầu JDK 26. Client có pom.xml và được build với release 17. Web là JavaScript thuần, không cần chạy Node để phục vụ giao diện. Node dùng cho kiểm thử frontend. JDK chứa javac để biên dịch; JVM thực thi bytecode; JAR đóng gói các class và tài nguyên.

Đọc mã nguồn: README mục 1, 2 và 6; client/src/main/java/vn/studyfix/client/.

3 TCP Socket và các khái niệm mạng

IP và cổng dùng để làm gì

IP xác định máy đích; cổng xác định dịch vụ trên máy đó. Có thể ví IP là địa chỉ trường và cổng là số phòng. Cùng một máy server có thể chạy nhiều dịch vụ ở cổng khác nhau. 127.0.0.1 hoặc localhost luôn chỉ chính máy đang truy cập, không phải máy của giáo viên ở đầu kia mạng.

TCP Socket là gì

Socket là đầu mút giao tiếp mạng mà chương trình sử dụng. Trong dự án, client tạo java.net.Socket để kết nối đến host và port. Server tạo ServerSocket để lắng nghe; accept() trả về một Socket riêng cho kết nối vừa được nhận. Socket của client và socket nhận ở server tạo thành hai đầu của một kết nối.

TCP cung cấp điều gì

TCP là luồng byte có kết nối, truyền đúng thứ tự và có cơ chế phát hiện mất gói, truyền lại. Hai bên có thể gửi và nhận trên cùng kết nối. Tuy nhiên, việc ghi dữ liệu vào socket không chứng minh nghiệp vụ đã được lưu trong database; muốn biết lưu thành công phải đọc phản hồi do ứng dụng trả về.

Ở mức khái niệm, kết nối TCP được thiết lập qua bắt tay SYN → SYN-ACK → ACK; Java và hệ điều hành thực hiện phần này khi connect và accept, nhóm không tự viết lại. Theo mô hình TCP/IP, lệnh StudyFix và HTTP thuộc tầng ứng dụng, TCP thuộc tầng vận chuyển, IP thuộc tầng Internet. UDP gửi datagram và không tự bảo đảm thứ tự hay truyền lại như TCP.

Vì sao chọn TCP cho code và chat

Code, lệnh và chat cần giữ nội dung và thứ tự. TCP phù hợp hơn việc tự xây cơ chế tin cậy trên UDP. Đổi lại, khi có mất gói, dữ liệu sau có thể phải chờ phần bị thiếu. Với audio/video thời gian thực, WebRTC có cách truyền và điều khiển media riêng, không nên suy luận mọi media trong dự án đều dùng TCP 5001.

Kênh	Cổng mặc định	Nội dung
TCP điều khiển	5000	Đăng nhập, phòng, chat, code chung
TCP màn hình Swing	5001	Packet loại và ảnh JPEG
TCP bài tập Swing	5002	Yêu cầu và phản hồi JSON
Web HTTP hoặc HTTPS	8080 mặc định	File giao diện, API và signaling

Đọc mã nguồn: TcpControlConnection.connect; Server.acceptLoop; README mục 3 và 8.

4 Giao thức riêng của nhóm

Nhóm dùng TCP có sẵn ở tầng vận chuyển và thiết kế quy tắc trao đổi ở tầng ứng dụng. Phần tự thiết kế là định dạng bản tin, tên lệnh, tham số, phản hồi và quy tắc quyền/trạng thái. JSON chỉ là định dạng biểu diễn dữ liệu, không phải cơ chế vận chuyển thay cho TCP.

Khung bản tin kênh điều khiển

```
[ 4 byte độ dài N theo big endian ][ N byte nội dung UTF-8 ]
Ví dụ payload: /ping
UTF-8: 2F 70 69 6E 67  →  N = 5
Toàn frame: 00 00 00 05 2F 70 69 6E 67
```

Big-endian nghĩa là byte có trọng số lớn đứng trước. Giới hạn payload là 65.536 byte. Độ dài phải tính theo số byte sau mã hóa UTF-8, không dùng số ký tự Java; chữ tiếng Việt có thể chiếm nhiều byte.

Tại sao cần 4 byte độ dài

TCP không giữ ranh giới từng lần gửi. Một lần gửi có thể được nhận thành nhiều lần đọc; nhiều lần gửi có thể được nhận liền nhau. Ví dụ gửi “ABC” rồi “DEF” không bảo đảm bên nhận đọc đúng hai lượt tương ứng. Header cho biết phải gom đủ bao nhiêu byte trước khi xử lý một bản tin.

Ví dụ các quy tắc ứng dụng

/login username password gửi thông tin đăng nhập. /join mã_phòng xin vào phòng. /chat nội_dung gửi chat. /request xin lượt sửa. /finish token hoàn tất nháp. Các marker như [LOGIN_OK], [LOGIN_FAILED] và [COMMITTED] giúp client nhận biết loại phản hồi. Kênh này dùng văn bản lệnh, không dùng JSON.

Hai kênh còn lại

Kênh 5002 cũng có header độ dài 4 byte, payload JSON gồm protocolVersion, requestId, type, sessionId và trường nghiệp vụ. Kênh 5001 có độ dài, sau đó là 1 byte loại packet và payload; các loại là HELLO, FRAME, STOP, ERROR, STATUS. Ảnh không gửi qua frame điều khiển 64 KiB.

Đọc mã nguồn: client/net/FrameCodec.java; server/Protocol.java; AssignmentConnection.java; screenshare/ScreenShareWire.java.

5 Đọc hiểu TcpControlConnection

Mở kết nối

```
Socket socket = new Socket();
socket.connect(new InetSocketAddress(host, port), timeoutMillis);
socket.setKeepAlive(true);
```

connect() có thời gian chờ để tránh đợi vô hạn khi chưa kết nối được. setKeepAlive(true) bật cơ chế keepalive của hệ điều hành. Ngoài ra ứng dụng có heartbeat /ping riêng, nên không được coi hai cơ chế này là một. setTcpNoDelay(true) tắt thuật toán Nagle, phù hợp các lệnh ngắn cần gửi sớm; nó không bảo đảm mạng không có độ trễ.

Một reader và một writer

send(message) kiểm tra kích thước, trạng thái kết nối rồi đưa lệnh vào outbound. Writer lấy lệnh theo thứ tự và gọi FrameCodec.write. Reader đọc frame, ghép lịch sử nếu nhiều phần, sau đó báo cho waiter và listener. Vì có một nơi đọc socket nên các module không tranh nhau lấy mất phản hồi.

```
Thao tác UI → Service → send → outbound → Writer → Socket
Socket → Reader → ghép frame → Listener → Controller → UI
```

Đọc ghi frame thực tế

```
int length = in.readInt();
// Kiểm tra 0 <= length <= 65536
byte[] payload = new byte[length];
in.readFully(payload);
String text = new String(payload, StandardCharsets.UTF_8);
```

readFully quan trọng vì read thông thường có thể trả ít byte hơn yêu cầu. Khi gửi, mã hóa UTF-8, ghi writeInt(payload.length), ghi payload rồi flush. Việc đồng bộ trên output giữ cả header và body không bị luồng khác ghi xen.

Chờ phản hồi và lỗi kết nối

sendAndAwait dùng Predicate để nhận diện phản hồi và CompletableFuture để đợi có timeout. Giao thức điều khiển chưa có request ID chung nên ghép theo điều kiện vẫn có giới hạn khi nhiều yêu cầu tương tự đồng thời. fail() đóng kết nối một lần, dừng heartbeat, báo lỗi cho yêu cầu đang đợi và thông báo listener.

Đọc mã nguồn: client/src/main/java/vn/studyfix/client/net/TcpControlConnection.java và FrameCodec.java.

6 Server đa luồng và đồng bộ dữ liệu

Server nhận nhiều người dùng thế nào

Server có acceptLoop chờ accept(). Mỗi socket được bọc trong ClientHandler rồi đưa vào executor. Reader của handler đọc liên tục các lệnh; writer lấy phản hồi từ hàng đợi để gửi. Giới hạn mặc định là 128 kết nối điều khiển, pool tối đa 256 worker. Đây là giới hạn cấu hình/mã nguồn, không phải kết quả đo tải thực tế.

Vì sao cần hàng đợi gửi

Nếu phát tin cho cả phòng mà trực tiếp chờ một máy đọc chậm, những người khác sẽ bị ảnh hưởng. Room chỉ đưa thông báo vào hàng đợi của client; writer chịu trách nhiệm truyền mạng. Server giới hạn hàng đợi 64 mục mỗi client và ngắt kết nối khi đầy để đồng bộ lại. Client Swing có outbound giới hạn 256 mục.

Tranh chấp dữ liệu là gì

Hai yêu cầu cùng đọc code phiên bản 10 rồi cùng lưu có thể làm mất thay đổi của nhau. Room dùng synchronized để các thao tác quan trọng trên cùng phòng được xử lý an toàn. ConcurrentHashMap hỗ trợ truy cập map đồng thời, nhưng không tự bảo vệ mọi chuỗi thao tác nhiều bước bên trong một phòng.

Khóa và phiên bản bảo vệ hai vấn đề khác nhau

Khóa ngăn các luồng sửa trạng thái RAM cùng lúc. expectedVersion phát hiện yêu cầu dựa trên dữ liệu cũ: nếu server đã ở v11 thì yêu cầu lưu dựa trên v10 bị từ chối. Token lượt sửa giúp từ chối yêu cầu của lượt đã hết hoặc của người không giữ lượt. Không gọi token lượt sửa là mật khẩu tài khoản.

Heartbeat và xử lý mất mạng

Client gửi /ping mỗi 5 giây; server trả /pong. Nếu thiếu heartbeat khoảng 20 giây, server đóng kết nối và dọn thành viên, quyền sửa, hàng đợi. Heartbeat là cách phát hiện phiên không còn hoạt động ở tầng ứng dụng. Timeout kết nối, timeout chờ phản hồi và heartbeat timeout là ba tình huống khác nhau.

Đọc mã nguồn: Server.java; ClientHandler.java; Room.java; TcpControlConnection.java.

7 Luồng đăng nhập vào phòng và chat

Đăng nhập bằng client Swing

Bước 1: client mở Socket đến server cổng 5000. Bước 2: AuthService gửi /login trong một frame. Bước 3: ClientHandler gọi Store kiểm tra tài khoản và mật khẩu. Bước 4: nếu hợp lệ, server gắn Identity vào kết nối và trả [LOGIN_OK] kèm session, userId, role. Nếu sai thì trả [LOGIN_FAILED]; sau 5 lần sai đóng phiên.

Đăng nhập trên web

Trình duyệt gửi HTTP đến gateway. Gateway mở kết nối TCP nội bộ đến server lõi để thực hiện đăng nhập, rồi quản lý phiên web và bearer token riêng. Những request HTTP tiếp theo gửi Authorization: Bearer token. Token web không phải token lượt sửa và không phải sessionId công khai của kết nối lõi.

Tạo và vào phòng

Giáo viên yêu cầu tạo phòng. Server kiểm tra vai trò và trạng thái chưa ở phòng khác, chuẩn hóa mã phòng rồi lưu dữ liệu. Khi tham gia, server xác nhận phòng hoạt động và gắn phiên vào danh sách thành viên. Mỗi phiên chỉ ở một phòng tại một thời điểm. Giáo viên khác không tự trở thành chủ phòng do có cùng vai trò TEACHER.

Luồng chat

Người dùng nhập tin → client gửi lệnh/API → server kiểm tra đăng nhập, thành viên và giới hạn nội dung → lấy danh tính người gửi từ phiên → lưu/phát thông báo theo luồng phòng → các client hiển thị. Không tin tên người gửi do client tự khai. Dữ liệu chat phòng học được lưu, còn mốc “đã đọc” của giao diện được giữ ở trình duyệt.

Phân biệt các loại định danh

Tên	Ý nghĩa	Ví dụ sử dụng
userId	Danh tính tài khoản lâu dài	Nhánh của sinh viên sau đăng nhập lại
sessionId	Một phiên/kết nối đăng nhập	Theo dõi thành viên online
roomId	Định danh nội bộ của phòng	Tham chiếu dữ liệu trong database
Mã phòng	Tên/mã để người dùng tìm và vào	Có thể chứa khoảng trắng, tiếng Việt

Đọc mã nguồn: ClientHandler.run; WebGatewayServer.WebSession.login; Room.java; RoomCode.java.

8 Sửa code chung theo lượt trên Swing

Luồng này dành cho cơ chế cộng tác code chung theo lượt. Cần phân biệt với nhánh cá nhân trên web. Không nên mô tả rằng mọi sinh viên đồng thời sửa trực tiếp cùng một bản Master bằng kỹ thuật giống Google Docs.

Chu trình một lượt sửa

```
Xin lượt → nhận quyền và token → chọn dòng theo version
→ sửa trong nháp → xem nháp → hoàn tất hoặc hủy
→ lưu revision thành công → thông báo code mới → nhả lượt
```

Nếu đã có người giữ lượt, /request đưa người tiếp theo vào hàng đợi FIFO. Người đang sửa dùng /select version token để chọn dòng hoặc đoạn. /setline, /delete, /keep hay /setrange mô tả thay đổi. Server ghép bản nháp; /draft xem kết quả chưa commit. /finish hoàn tất khi đúng quyền, token, version và đủ thao tác. /cancel bỏ nháp và nhả lượt.

Ví dụ tránh ghi đè

An nhận lượt ở version 8 với token của lượt hiện tại. Khi An gửi lệnh hoàn tất, server kiểm tra An còn giữ lượt và nền dữ liệu chưa đổi. Nếu quyền đã bị thu hồi, token cũ không còn hợp lệ. Nếu version không khớp, server yêu cầu lấy lại trạng thái. Nhờ đó phản hồi mạng chậm từ lượt trước không ghi đè code mới.

Lưu thành công cần theo thứ tự nào

Server ghi code và revision trong database trước. Khi commit thành công mới cập nhật trạng thái RAM và đưa thông báo vào hàng đợi gửi. Nếu lưu thất bại, không báo thành công và không tăng trạng thái như đã lưu. Bản nháp chưa hoàn tất không tự trở thành code chung.

Lịch sử và diff

Revision giữ snapshot, tác giả, thời điểm, phiên bản trước/sau và diff. Snapshot là toàn bộ nội dung tại một thời điểm; diff là phần thay đổi để đọc nhanh. Xem lịch sử không tự khôi phục code. Lịch sử dài được chia thành [HISTORY_PART i/n] và ghép đủ ở client, không cắt giữa ký tự Unicode.

CodeDiff bỏ phần đầu/cuối giống nhau rồi dùng LCS cho vùng nhỏ hoặc Myers có giới hạn cho vùng lớn. LCS tìm dãy con chung dài nhất để suy ra dòng thêm/xóa. Khi vượt ngân sách xử lý, hệ thống biểu diễn thay toàn đoạn giữa để tránh tốn tài nguyên không giới hạn.

Giới hạn cần nhớ

Source nghiệp vụ tối đa 32.000 byte UTF-8. Khi phòng đã sử dụng phiên Master/nhánh, server kiểm tra để sinh viên không lách về lệnh TCP cũ nhằm sửa Master. Chỉ ẩn nút trên web không đủ để thực thi quy tắc này.

Đọc mã nguồn: Room.java; EditSession.java; CodeRevision.java; Protocol.historyFrames; HistoryFrameAssembler.java.

9 Phòng học Master và nhánh cá nhân

Master là code chung phục vụ giảng bài. Branch là bản làm riêng của mỗi sinh viên trong một vòng học. Fork tạo các bản riêng từ cùng một snapshot Master; Freeze dừng sửa để giáo viên xem; Merge trong dự án là chọn toàn bộ code của một nhánh làm Master mới.

```
WAITING → START → LECTURE → FORK → BRANCHING
BRANCHING → FREEZE → MERGING → MERGE → LECTURE
MERGING → RESUME → BRANCHING
Phiên đang học → END → ENDED
```

Luồng từ góc nhìn người sử dụng

Giáo viên bắt đầu, chọn bài DEBUG_COLLAB và lưu Master. Khi có sinh viên, giáo viên Fork từ đúng version. Mỗi sinh viên sửa nhánh của mình, lưu và yêu cầu chấm code đã lưu. Giáo viên xem danh sách và chi tiết từng bài, Freeze rồi chọn nhánh làm Master. Sau đó quay về giảng trên Master mới hoặc mở vòng tiếp.

Luồng từ góc nhìn server

ClassroomService kiểm tra thành viên, vai trò chủ phòng, phase hiện tại, round và expectedVersion. Nhánh gắn với phòng, vòng và userId. BRANCH_SAVE chỉ chấp nhận nhánh của chính sinh viên trong BRANCHING. MERGE chỉ chấp nhận ở MERGING và Master còn đúng nền của vòng. Repository lưu thay đổi rồi Room công bố trạng thái mới.

Ví dụ cụ thể

Master v5 được tách thành nhánh An và Bình ở vòng 2. An lưu nhánh lên version 3; Bình lưu version 2. Giáo viên Freeze và chọn An. Server dùng toàn bộ snapshot nhánh An để tạo revision Master mới. Code của Bình vẫn là dữ liệu nhánh/lịch sử vòng đó; hệ thống không tự hòa trộn dòng của An với Bình như Git.

Các câu hỏi dễ nhầm

Merge không bắt buộc nhánh phải AC. Sinh viên vào muộn trong BRANCHING nhận nhánh từ nền của vòng; vào trong MERGING chờ vòng sau. Nhánh đã lưu có thể lấy lại sau đăng nhập lại cùng tài khoản. Kết quả chấm gắn với version cụ thể; nếu code đã đổi thì kết quả cũ được đánh dấu không còn tương ứng.

Đọc mã nguồn: server/src/studyfix/ClassroomService.java và ClassroomRepository.java; README mục 4.2.

10 Cơ sở dữ liệu và tính nhất quán

Database	Nhóm dữ liệu	Lớp liên quan
studyfix.db	Tài khoản, phòng, revision, phiên học, nhánh, chat, kết quả phòng học	Store, ClassroomRepository
studyfix-learning.db	Đề, bài giao, bài nộp, luyện tập, cuộc thi, lượt chạy giáo viên	AssignmentRepository, PracticeRepository

SQLite và JDBC

SQLite là hệ quản trị cơ sở dữ liệu nhúng, lưu dữ liệu trong file. Không cần một tiến trình database server riêng để demo. Java gọi API JDBC thông qua driver SQLite JDBC. Ưu điểm là triển khai gọn; đổi lại cần quản lý ghi đồng thời, sao lưu và dung lượng lịch sử phù hợp, không mặc định coi nó đáp ứng mọi quy mô lớp học.

RAM khác dữ liệu lưu bền

RAM giữ kết nối online, trạng thái và dữ liệu đang dùng để xử lý nhanh. Database giữ những dữ liệu cần tồn tại sau restart. Khi server khởi động lại, phòng và revision có thể tải lại nhưng socket cũ đã mất; người dùng cần đăng nhập và vào lại phòng. Nháp trong trình duyệt có cơ chế lưu riêng, không được coi là luôn đã lưu lên server.

Transaction là gì

Transaction gom nhiều thao tác thành một đơn vị. Ví dụ Merge cần ghi Master, revision, phase và sự kiện nhất quán. Nếu chỉ một phần được lưu, lớp học có thể thấy Master mới nhưng trạng thái vòng cũ. Commit xác nhận toàn bộ; rollback hủy các thay đổi của giao dịch khi thất bại. Sau commit mới cập nhật RAM và báo thành công.

Schema migration và xóa phòng

Migration là nâng cấu trúc database khi phiên bản phần mềm thay đổi. Hai database có schema riêng. Xóa phòng là đánh dấu inactive và giữ lịch sử, không xóa sạch mọi bảng liên quan. Tạo lại cùng mã phòng tạo ID mới, không nhập nhằng với lịch sử phòng cũ.

Sao lưu khi chuẩn bị demo

Dừng server và chờ đóng database trước khi sao chép bộ server/data. Nếu có file -wal, -shm thì giữ cùng bộ dữ liệu. Không xóa database để sửa lỗi đăng nhập. Một tiến trình server sở hữu bộ database đang chạy; tránh chạy hai launcher trên cùng cổng và dữ liệu.

Đọc mã nguồn: Store.java; ClassroomRepository.java; assignment/persistence/; README mục 10.

11 Luồng nộp bài và chấm bằng Docker

```
Chọn bài → gửi source và thông tin yêu cầu → xác thực quyền
→ lưu snapshot → QUEUED → worker → RUNNING
→ Docker biên dịch và chạy test → lưu kết quả → client đọc kết quả
```

Vì sao phải lưu snapshot trước

Snapshot ghi nhận chính xác code được nộp. Người dùng sửa tiếp editor cũng không đổi nội dung đang được chấm. Vì vậy lịch sử và kết quả luôn có thể liên kết đến đúng phiên bản. Chấm nhánh phòng học cũng lưu version của nhánh tại lúc gửi, tránh gán kết quả cũ cho code mới.

Hàng đợi và worker

Việc biên dịch/chạy lâu hơn xử lý một request mạng. Server đưa công việc vào queue để request không phải giữ toàn bộ xử lý trực tiếp. Module bài tập/luyện tập/cuộc thi dùng queue mặc định 32 và 2 worker. Phòng học có queue chấm riêng 32, 1 worker. Khi hàng đợi đầy phải báo lỗi rõ ràng, không giả vờ đã chấm thành công.

Sandbox bảo vệ server thế nào

DockerJavaSandbox tạo môi trường container để biên dịch Main.java và chạy từng test. Môi trường có giới hạn mạng, CPU, RAM, PID, thời gian và output. Filesystem gốc chỉ đọc; thư mục biên dịch/chạy được gắn theo nhu cầu. Input được gửi qua stdin; đáp án kỳ vọng ở server. Test ẩn và đáp án không nằm trong thư mục bài nộp truy cập được.

Phân biệt các kết quả

COMPILE_ERROR là lỗi biên dịch. RUNTIME_ERROR là chương trình chạy bị lỗi. WRONG_ANSWER là output không đúng. TIME_LIMIT_EXCEEDED là quá thời gian. ACCEPTED nghĩa là bài đáp ứng các test và quy tắc chấm tương ứng. QUEUED, RUNNING hoặc SANDBOX_UNAVAILABLE không phải AC. Nếu Docker/image không sẵn sàng, hệ thống báo lỗi môi trường, không tự chạy code sinh viên trong JVM server.

Chạy thử khác nộp chính thức

Luyện tập chạy thử chỉ dùng test công khai và không tạo phiên bản nộp chính thức. Nộp bài lưu snapshot và chấm đầy đủ test. Idempotency key ràng buộc yêu cầu và source để khi gửi lại cùng bài nộp không vô tình tạo thêm phiên bản. requestId nhận diện yêu cầu không mặc định là cùng khái niệm chống trùng nghiệp vụ.

Đọc mã nguồn: assignment/service/AssignmentService.java; grading/DockerJavaSandbox.java; ClassroomService.java; README mục 9.

12 Luyện tập giao bài và cuộc thi

Ba ngữ cảnh khác nhau

Luyện tập là người dùng tự chọn bài, không cần vào phòng hoặc được giáo viên giao. Bài được giao gắn với quyền của giáo viên chủ phòng, danh sách người nhận và hạn nộp. Cuộc thi có tập bài và thời gian bắt đầu/kết thúc riêng. Cả ba tái sử dụng cơ chế snapshot, hàng đợi và bộ chấm nhưng phải giữ đúng người, bài và ngữ cảnh khi tính kết quả.

Luồng luyện tập cá nhân

Người dùng mở danh sách, tìm/chọn bài, đọc đề và code mẫu, sửa trong editor. Chạy thử dùng test công khai. Nộp bài tạo snapshot chính thức và phiên bản mới; lịch sử riêng có 10 bản mỗi trang. Nháp localStorage được tách theo tài khoản và bài. Nạp file .java hoặc .txt tối đa 32.000 byte cần xác nhận trước khi thay nội dung và không tự nộp.

Đã AC nghĩa là tài khoản từng có bài nộp được chấp nhận cho bài đó; lần nộp mới sai không xóa thành tích cũ. Tuy nhiên mỗi snapshot vẫn giữ verdict riêng. Giáo viên không tự có quyền xem code luyện riêng của người khác chỉ vì mang vai trò giáo viên.

Luồng giao bài

Giáo viên chủ phòng chọn đề trong ngân hàng, xác định cả phòng hoặc một số người nhận, đặt hạn rồi giao. Danh sách người nhận được chụp tại thời điểm giao, tránh việc sinh viên vào muộn làm thay đổi báo cáo cũ. Sinh viên xem phần được phép, làm và nộp snapshot; server kiểm tra người nhận, hạn và loại bài. Giáo viên xem kết quả trong đúng phạm vi quản lý.

Luồng cuộc thi

Giáo viên tạo cuộc thi từ ID các đề trong ngân hàng, không bắt buộc giao bài trước. Server kiểm tra thời gian bắt đầu/kết thúc khi xử lý nghiệp vụ. Loại INDIVIDUAL chỉ nhận bài BUILD_FROM_SCRATCH; TEAM_COLLAB chưa được hỗ trợ. Bảng xếp hạng web hiện dành cho giáo viên và có kiểm tra quyền ở API. Lượt chạy thử riêng của giáo viên không tính vào thống kê sinh viên hoặc điểm thi.

Ví dụ câu trả lời khi được hỏi điểm khác nhau

“Cùng một bộ chấm có thể phục vụ nhiều chức năng, nhưng quyền truy cập, thời hạn và cách thống kê khác nhau. Vì vậy server phải gắn mỗi bài nộp với đúng tài khoản, đúng đề và đúng ngữ cảnh, không chỉ nhìn thấy một kết quả AC rồi tính cho mọi bài.”

Đọc mã nguồn: assignment/service/AssignmentService.java; assignment/persistence/; features/exercises/ và features/contests/; README mục 4.3 và 4.4.

13 Giao diện web API và đồng bộ

Cách frontend được tổ chức

index.html nạp app.js bằng type="module". core/state.js giữ trạng thái; core/api.js bao fetch và lỗi; core/router.js đăng ký trang; core/events.js nối thao tác UI với chức năng. features chia theo auth, room, classroom, screen-share, exercises và contests. ui chứa các thành phần dùng chung.

Một thao tác HTTP đi qua đâu

```
Nhấn nút → hàm trong features → core/api.js → HTTP request
→ WebGatewayServer → kiểm tra phiên và quyền → dịch vụ
→ lưu dữ liệu nếu cần → JSON response → cập nhật giao diện
```

api.js tự gắn bearer token khi có phiên, chuyển body thành JSON và đọc response JSON. HTTP 401 khi phiên không hợp lệ kích hoạt xử lý đăng xuất. Các lỗi khác hiển thị thông điệp phù hợp. API bài tập dùng /api/assignments; lệnh phòng dùng /api/rooms/current/command.

Polling là gì

Polling là client chủ động hỏi server định kỳ để lấy trạng thái/sự kiện mới. Trong dự án, polling dùng để đồng bộ dữ liệu và nhận signaling. Vì vậy không mô tả web đang dùng WebSocket cho mọi sự kiện. Giao diện cần kiểm tra phản hồi còn thuộc phòng, bài hoặc vòng đang mở trước khi áp dụng để tránh response muộn ghi đè màn hình mới.

Ba nơi lưu trạng thái trên trình duyệt

Biến JavaScript trong RAM mất khi tải lại. sessionStorage giữ nháp theo tab trong luồng phòng học và có thể mất khi đóng tab. localStorage giữ nháp luyện tập theo tài khoản/bài lâu hơn. Các kho này hỗ trợ trải nghiệm, còn bản lưu chính thức và kết quả chấm do server/database quản lý.

Tách module giúp bảo trì thế nào

Chức năng riêng có nơi sửa rõ ràng, tránh dồn tất cả vào app.js. Trang đăng ký qua registerPage và callback giúp giảm import vòng. HTTP/JSON dễ tích hợp với trình duyệt; gateway tái sử dụng server lõi và module nghiệp vụ thay vì cho web tự quyết định quyền.

Đọc mã nguồn: web-client/static/core/api.js, polling.js, router.js; WebGatewayServer.java; README mục 6.3 và 7.

14 Media WebRTC và màn hình Swing

WebRTC trên trình duyệt

WebRTC là công nghệ thiết lập kết nối truyền media giữa các peer. Trình duyệt xin quyền mic/camera hoặc chia sẻ màn hình, tạo track và dùng RTCPeerConnection để trao đổi. Dự án dùng mô hình mesh trực tiếp trong LAN: mỗi người có thể có các kết nối tới người khác.

Signaling không phải media

Trước khi truyền media, các peer trao đổi OFFER, ANSWER và ICE qua signaling của server. OFFER/ANSWER mang mô tả phiên SDP; ICE mang ứng viên đường kết nối. Có thể ví signaling là trao đổi cách liên lạc, còn media là cuộc trò chuyện sau khi đã nối. Gateway kiểm tra các bên chủ động tham gia và cùng phòng trước khi chuyển tín hiệu.

Âm thanh và chia sẻ màn hình

Mic và âm thanh tab là hai nguồn khác nhau. Tắt mic không tắt âm thanh tab. Để chia sẻ video kèm tiếng, thường cần chọn tab trình duyệt và bật chia sẻ âm thanh tab. Máy nhận đôi khi phải bấm phát vì chính sách tự phát của trình duyệt. Quyền thiết bị và kết nối LAN cần được kiểm tra trên máy demo thật.

Đường chia sẻ màn hình của Swing

```
Xin screen ticket qua TCP 5000 → nối relay TCP 5001
→ HELLO kèm vé → chụp màn hình → nén JPEG → FRAME
→ relay chuyển đến người xem được phép → giải mã và hiển thị
```

Vé có hạn 30 giây và dùng một lần, gắn với phiên, phòng, chế độ PUBLISH/VIEW. Packet có body tối đa 2 MiB. Relay này truyền ảnh JPEG; không gọi nó là toàn bộ hệ thống gọi WebRTC của web. Mỗi loại client có giải pháp media riêng.

Giới hạn triển khai

Web giới hạn 16 thiết bị media mỗi phòng trong logic, chưa chứng minh tải 16 camera thật. Chưa có SFU/TURN và chưa bảo đảm kết nối qua NAT/mạng khác nhau. HTTP localhost có thể dùng khi phát triển; truy cập LAN để dùng thiết bị cần HTTPS/chứng chỉ tin cậy theo cấu hình. Media và signaling cuộc gọi không được lưu vào database.

Đọc mã nguồn: features/classroom/call.js; features/screen-share/screen-share.js; MeetingCalls.java; screenshare/.

15 Hướng đối tượng và bảo vệ dữ liệu

Đóng gói và phân chia trách nhiệm

TcpControlConnection giấu socket, luồng đọc/ghi và hàng đợi sau các phương thức như send. FrameCodec chỉ tập trung chuyển thông điệp thành frame và ngược lại. Room tập trung quy tắc phòng. Sự tách biệt này giúp giao diện gọi chức năng mà không cần tự biết từng byte truyền mạng.

Interface và tính đa hình

ClientModule quy định tabTitle() và view(). Một module như ExerciseModule cung cấp cách triển khai để ghép vào giao diện. GradingSandbox mô tả khả năng chấm; DockerJavaSandbox là cách triển khai thực tế. Interface là hợp đồng, còn class là phần thực hiện. ClientModule không phải lớp giao thức mạng.

Composition và sự kiện

RoomController giữ RoomService và CodeCollaborationService để phối hợp, thay vì phải kế thừa mọi chức năng. ConnectionListener cho phép nhiều nơi được báo khi có thông điệp hoặc mất kết nối. Đây là cách tổ chức theo sự kiện tương tự ý tưởng Observer. Không cần gán nhãn toàn bộ dự án là MVC thuần; có thể nói client tách UI, controller, model và service theo trách nhiệm.

Luồng giao diện Swing

Swing có Event Dispatch Thread để cập nhật UI. Chờ mạng hay chạy I/O lâu trên luồng này sẽ làm cửa sổ đơ. Dự án có SwingExecutor dùng invokeLater để chuyển cập nhật về EDT. Luồng mạng nhận dữ liệu, controller cập nhật trạng thái và giao diện phản ánh lại qua cơ chế phù hợp.

Xác thực khác phân quyền

Xác thực trả lời “bạn là ai”, ví dụ kiểm tra username/password. Phân quyền trả lời “bạn được làm gì”, ví dụ chỉ chủ phòng được Merge hoặc xóa phòng. Server lấy danh tính từ phiên đăng nhập; không tin role hay userId tự gửi. Sinh viên chỉ xem nhánh/luyện tập riêng theo quy tắc nghiệp vụ.

Mật khẩu và mã hóa đường truyền

Passwords dùng PBKDF2-HMAC-SHA256 với 600.000 vòng, salt ngẫu nhiên 16 byte và đầu ra 256 bit. Đây là băm dẫn xuất mật khẩu, không phải mã hóa có thể giải ngược. Salt giúp hai người cùng mật khẩu có giá trị lưu khác nhau. Băm trong database không làm TCP thường được mã hóa; HTTPS chỉ bảo vệ đường web đã bật TLS, không tự bảo vệ các cổng TCP riêng.

Đọc mã nguồn: ClientModule.java; ExerciseModule.java; GradingSandbox.java; SwingExecutor.java; Passwords.java.

16 Kiểm thử giới hạn và hướng phát triển

Nói thế nào về kiểm thử

Dự án có suite kiểm thử Java cho TCP, lưu trữ, phòng học, quyền, recovery, chat, cuộc gọi signaling và module bài tập. Frontend có kiểm tra import/module và các luồng UI mô phỏng. Điều này cho thấy có cơ chế kiểm tra; chỉ được tuyên bố "đã chạy đạt" khi có kết quả chạy tương ứng trên phiên bản và môi trường cụ thể.

Tình huống	Kết quả cần kiểm tra
Hai client cùng sửa từ version cũ	Server từ chối yêu cầu không khớp version
Sinh viên gửi yêu cầu Merge	Server từ chối do không có quyền chủ phòng
Ngắt mạng người giữ lượt	Phiên bị dọn, lượt không bị kẹt vô hạn
Database ghi lỗi	Không thông báo đã commit thành công
Docker tắt hoặc thiếu image	Báo sandbox không sẵn sàng, không trả AC
Server restart	Dữ liệu đã lưu còn, kết nối cần đăng nhập lại
Đổi bài trong lúc request đang chạy	Không gán phản hồi cũ vào bài mới

Giới hạn nên chủ động trình bày

Giao thức text điều khiển chưa có request ID chung cho mọi phản hồi. Lịch sử code chưa phân trang ở database và được nạp vào RAM. Media mesh tăng số kết nối khi lớp đông; chưa có SFU/TURN hay đo tải lớp đông thật. Chưa có thi đội đầy đủ, chống gian lận hoặc tự đăng nhập lại sau server restart.

Hướng cải tiến có lý do

Có thể chuẩn hóa envelope JSON và request ID cho kênh điều khiển để ghép phản hồi rõ hơn. Bổ sung phân trang lịch sử để giảm RAM. Bổ sung TLS cho TCP nếu triển khai ngoài môi trường tin cậy. Khi mở rộng media, nghiên cứu TURN cho kết nối qua mạng và SFU cho phân phối media. Đây là đề xuất, không trình bày như chức năng hiện có.

Đọc mã nguồn: README mục 11 và 12; server/test/; web-client/tests/.

17 Kịch bản demo trên lớp

Chuẩn bị trước khi vào lớp

1. Sao lưu dữ liệu khi server đã dừng. Chuẩn bị sẵn JDK/Ant; bật Docker và có image eclipse-temurin:21-jdk nếu sẽ chấm thật. Không đợi tải image khi đang thuyết trình.

2. Từ thư mục StudyFix chạy server/run-server-manager.ps1 bằng PowerShell. Chọn một launcher đầy đủ, tránh mở thêm server khác. Kiểm tra các dịch vụ 5000, 5001, 5002 và web 8080 đang sẵn sàng.

3. Mở http://localhost:8080. Dùng hai hồ sơ trình duyệt hoặc cửa sổ riêng tư để đăng nhập teacher và student1 tách phiên. Nếu demo Swing, build trước rồi chạy run-client-only.ps1. Chỉ dùng tài khoản demo nếu database hiện tại có các tài khoản đó.

Demo chính khoảng 6 phút

Phút 1: giáo viên tạo phòng, sinh viên tham gia. Giải thích server quản lý phòng và vai trò. Phút 2: chọn bài DEBUG_COLLAB, lưu Master, chỉ ra version. Phút 3: Fork, sinh viên sửa và lưu nhánh. Phút 4: chạy kiểm tra và giải thích QUEUED/RUNNING/verdict. Phút 5: giáo viên Freeze, xem nhánh và Merge. Phút 6: xem Master mới, lịch sử và chat.

Nếu thầy muốn xem TCP trực tiếp

Mở TcpControlConnection.connect để chỉ new Socket và connect. Mở FrameCodec để chỉ header độ dài, UTF-8, readFully. Mở Server.acceptLoop để chỉ ServerSocket.accept. Mở ClientHandler.run để chỉ /login và /ping. Có thể demo chat trên hai Swing client; không cần tìm giữa toàn bộ dự án lúc đang trả lời.

Phương án khi có lỗi môi trường

Nếu Docker lỗi, chỉ rõ SANDBOX_UNAVAILABLE là lỗi dịch vụ và tiếp tục demo lưu snapshot/lịch sử; không nói bài đã AC. Nếu mic/camera lỗi quyền hoặc HTTPS, chuyển sang phần kiến trúc signaling và demo trên localhost đã kiểm tra. Nếu cổng bận, dừng tiến trình server cũ bằng công cụ quản lý đang dùng rồi chạy lại; không xóa database.

Các file nên ghim sẵn trong IDE

Server.java, ClientHandler.java, Protocol.java, Room.java, ClassroomService.java, TcpControlConnection.java, FrameCodec.java, DockerJavaSandbox.java và core/api.js. Ghi nhớ đường đi đến file quan trọng sẽ giúp câu trả lời có bằng chứng.

18 Bài nói mẫu khoảng 5 đến 7 phút

Mở đầu khoảng 40 giây

“Thưa thầy và các bạn, StudyFix là ứng dụng hỗ trợ học và thực hành Java theo nhóm. Điểm chính là giáo viên giảng trên một bản code chung, cho sinh viên làm nhánh riêng rồi chọn một lời giải để tiếp tục giảng. Bên cạnh đó hệ thống có chat, luyện tập, bài giao và cuộc thi.”

Kiến trúc khoảng 1 phút

“Hệ thống theo mô hình client–server. Server viết bằng Java, lưu dữ liệu bằng SQLite, có giao diện web và client Swing. Các dịch vụ server chạy dưới dạng module trong cùng JVM. Client Swing dùng TCP 5000 cho điều khiển, 5001 cho ảnh màn hình và 5002 cho bài tập. Trình duyệt dùng HTTP hoặc HTTPS qua web gateway; media dùng WebRTC.”

Giao thức khoảng 1 phút

“Nhóm không tự tạo TCP mà thiết kế giao thức tầng ứng dụng trên TCP. Với kênh điều khiển, một bản tin gồm 4 byte độ dài và payload UTF-8. Cần header vì TCP là luồng byte, không giữ ranh giới từng bản tin. Bên nhận đọc độ dài, kiểm tra giới hạn rồi đọc đủ payload. Các lệnh gồm login, join, chat và các thao tác sửa code.”

Luồng cộng tác khoảng 1 phút rưỡi

“Sau khi đăng nhập, server gắn danh tính với phiên và kiểm tra quyền theo từng yêu cầu. Trên web, giáo viên lưu Master rồi Fork thành nhánh cho sinh viên. Mỗi người sửa nhánh riêng, server kiểm tra vòng và phiên bản khi lưu. Giáo viên Freeze và chọn một nhánh làm Master mới. Merge ở đây là chọn toàn bộ snapshot, không phải tự ghép nhiều nhánh như Git. Swing còn có cơ chế sửa code chung theo lượt và token.”

Dữ liệu và chấm bài khoảng 1 phút

“Khi lưu, hệ thống commit database trước rồi mới cập nhật RAM và báo thành công. Khi nộp bài, code được chụp snapshot, vào hàng đợi và được worker chấm bằng Docker. Sandbox giới hạn tài nguyên và không cho code sinh viên chạy trực tiếp trong JVM server. Kết quả gắn với đúng snapshot; lỗi Docker khác với bài sai.”

Đánh giá khoảng 40 giây

“Ưu điểm của thiết kế là tách trách nhiệm giữa giao diện, kết nối, nghiệp vụ và lưu trữ; quyền được kiểm tra ở server. Giới hạn hiện tại là giao thức điều khiển chưa có request ID chung, lịch sử chưa phân trang database và media chưa được đo tải lớp đông. Hướng phát triển là chuẩn hóa giao thức, tối ưu lịch sử và cải thiện triển khai media.”

Khi trình bày, hãy nói bằng lời của mình và chỉ vào màn hình/sơ đồ tương ứng. Dùng bài mẫu để giữ mạch, tránh học thuộc từng chữ rồi mất nhịp khi thầy hỏi chen.

19 Vấn đáp về kiến trúc và mạng

1 Nhóm có dùng TCP Socket không

Có. Client Swing dùng java.net.Socket; server dùng ServerSocket.accept. Bằng chứng rõ nhất là TcpControlConnection.connect và Server.acceptLoop. Trình duyệt giao tiếp qua HTTP/HTTPS thay vì trực tiếp mở java.net.Socket.

2 Nhóm tự thiết kế giao thức nào

Nhóm thiết kế giao thức tầng ứng dụng: header độ dài, nội dung lệnh, tham số và marker phản hồi trên TCP. Kênh bài tập dùng envelope JSON riêng. TCP, HTTP, JSON và WebRTC là công nghệ có sẵn.

3 Tại sao không chỉ gửi chuỗi rồi đọc một lần

TCP không giữ ranh giới bản tin. Bên nhận có thể đọc thiếu hoặc đọc nhiều bản tin liền nhau. Vì vậy phải có quy tắc đóng khung và đọc đủ số byte của payload.

4 Tại sao tính byte mà không tính ký tự

Header mô tả số byte sẽ đọc trên mạng. UTF-8 biểu diễn một ký tự bằng một hoặc nhiều byte, nhất là tiếng Việt. Dùng text.length() có thể làm bên nhận đọc lệch frame.

5 ServerSocket khác Socket ở đâu

ServerSocket lắng nghe trên một cổng. Mỗi lần accept nhận được một Socket để trao đổi với một client. Server vẫn giữ listener để tiếp tục nhận client khác.

6 Vì sao dùng nhiều cổng

Tách lệnh điều khiển, ảnh lớn và bài tập giúp mỗi kênh có định dạng, giới hạn và vòng đời phù hợp; module bài tập không tranh reader của điều khiển. Điều này không làm các kênh hoàn toàn độc lập về băng thông hoặc tài nguyên.

7 Hệ thống có phải microservices không

Các thành phần được chia module nhưng launcher hiện chạy chung một JVM. Vì vậy em mô tả là server modular theo mô hình client–server, chưa gọi là các microservice triển khai độc lập.

8 Có dùng WebSocket không

Luồng web trong mã nguồn dùng HTTP fetch và polling cho API/trạng thái/signaling. Cuộc gọi dùng WebRTC. Không nên gọi WebRTC là WebSocket hoặc khẳng định web đang dùng WebSocket nếu không có triển khai đó.

20 Vấn đáp về dữ liệu và đồng thời

9 Hai người cùng sửa thì xử lý thế nào

Swing dùng quyền sửa theo lượt và token. Web cho mỗi sinh viên nhánh riêng. Khi lưu, server kiểm tra version; thao tác quan trọng trong cùng phòng có khóa đồng bộ để bảo vệ trạng thái RAM.

10 Khóa rồi tại sao vẫn cần version

Khóa bảo vệ luồng chạy đồng thời trong server. Version phát hiện client gửi yêu cầu từ dữ liệu cũ, kể cả khi yêu cầu tới sau và không còn chạy cùng lúc. Hai cơ chế xử lý hai nguyên nhân khác nhau.

11 Merge có giống Git không

Không hoàn toàn. Trong StudyFix, giáo viên chọn toàn bộ snapshot của một nhánh làm Master mới. Không tự động ghép các thay đổi từ nhiều nhánh và không mặc định giải quyết xung đột theo Git.

12 Vì sao ghi database trước khi báo thành công

Nếu báo trước mà ghi thất bại thì UI và RAM sẽ cho rằng có dữ liệu chưa tồn tại bền vững. Commit trước giúp thành công ở giao diện tương ứng với thay đổi thực sự đã lưu.

13 Restart server có mất hết không

Dữ liệu đã lưu trong SQLite như phòng, code, revision và nhánh còn theo thiết kế lưu trữ. Kết nối online và phiên cũ không tự sống lại; người dùng phải đăng nhập và vào lại phòng. Nháp chưa lưu phụ thuộc nơi giữ nháp.

14 Tại sao dùng hai database

Dữ liệu lõi lớp học và dữ liệu learning được tách theo module. Mỗi database có schema và repository tương ứng. Không có nghĩa SQLite bắt buộc phải tách hai file hoặc hệ thống có transaction chung xuyên hai database.

15 Snapshot khác diff như thế nào

Snapshot giữ toàn bộ code tại thời điểm lưu/nộp, phù hợp khôi phục nội dung và chấm đúng bản. Diff mô tả phần thay đổi để đọc lịch sử dễ hơn. Một revision có thể giữ cả hai.

16 Hàng đợi và thread giúp gì

Reader không phải chờ giao diện, writer xử lý truyền riêng, còn queue chấm tách công việc lâu khỏi request. Queue có giới hạn giúp tránh tích lũy vô hạn. Nhiều thread không tự làm dữ liệu an toàn nên vẫn cần đồng bộ.

21 Vấn đáp về chấm bài và bảo mật

17 Tại sao dùng Docker

Code người học có thể lặp vô hạn, tạo nhiều output hoặc truy cập tài nguyên. Container cùng giới hạn mạng, thời gian, CPU, RAM và PID tạo môi trường chấm tách biệt hơn. Không nên tuyên bố sandbox là bảo đảm an toàn tuyệt đối.

18 Kết quả chấm có theo code đang mở không

Kết quả theo snapshot và version khi gửi chấm. Nếu người dùng sửa editor sau đó, kết quả cũ không tự đại diện cho code mới. Đây là lý do lưu snapshot và đánh dấu kết quả cũ khi version thay đổi.

19 Tắt Docker thì bài có bị chấm sai không

Hệ thống cần báo sandbox không sẵn sàng hoặc lỗi hệ thống. Không được coi đó là ACCEPTED và cũng không được quy thành lời giải sai. Bộ chấm không fallback chạy code sinh viên trong JVM server.

20 Ẩn nút Merge đã đủ phân quyền chưa

Chưa. Client có thể bị sửa hoặc gửi request thủ công. Server phải kiểm tra phiên, chủ phòng, phase, round và version trước khi chấp nhận Merge.

21 Băm mật khẩu có mã hóa đường truyền không

Không. PBKDF2 bảo vệ cách lưu mật khẩu trong database. TCP thông thường vẫn không mã hóa nội dung ứng dụng. HTTPS chỉ bảo vệ đường web đã cấu hình TLS; các cổng TCP khác cần giải pháp riêng nếu mở rộng triển khai.

22 Token web token sửa và vé màn hình khác nhau thế nào

Bearer token xác thực phiên HTTP. Token sửa ràng buộc lượt sửa code. Vé màn hình cấp quyền kết nối relay một lần, có hạn và gắn với phòng/chế độ. Không dùng thay thế lẫn nhau.

23 Tại sao phải kiểm tra ở server nếu UI đã kiểm tra

Kiểm tra ở client giúp phản hồi nhanh nhưng người dùng có thể bỏ qua. Kiểm tra ở server bảo vệ dữ liệu thực sự, đặc biệt quyền xem nhánh, quyền chủ phòng, thời hạn nộp và giới hạn source.

24 Làm sao biết chấm thật chứ không phải mô phỏng

Cần chạy Docker Engine và image thật, nộp chương trình đúng/sai/lỗi rồi quan sát verdict, snapshot và log tương ứng. Suite mô phỏng hoặc test lưu trữ đạt không tự chứng minh bộ chấm Docker thật đã hoạt động.

22 Vấn đáp nâng cao và ôn nhanh

25 WebRTC khác signaling ở đâu

Signaling chuyển thông tin OFFER/ANSWER/ICE để thiết lập và điều khiển kết nối. Media là audio/video truyền giữa peer theo mô hình WebRTC hiện có. Server signaling không nhất thiết vận chuyển toàn bộ media.

26 Vì sao nhiều người gọi cùng lúc có thể nặng

Mesh làm mỗi người phải kết nối tới nhiều người khác. Số cặp kết nối tối đa giữa n người là n(n−1)/2. Đây là lý do cần đo tải và cân nhắc SFU khi tăng quy mô; giới hạn 16 trong code không chứng minh 16 camera chạy mượt.

27 Heartbeat có thay được ACK lưu bài không

Không. Heartbeat xác nhận ứng dụng/kết nối còn hoạt động. Phản hồi nghiệp vụ xác nhận yêu cầu cụ thể được chấp nhận hoặc lưu thành công. TCP ACK cũng không đồng nghĩa database đã commit.

28 Tại sao kết quả request cũ gây lỗi UI

Người dùng có thể đổi phòng/bài trước khi response trở về. Nếu áp response vào trạng thái hiện tại mà không đối chiếu, code hay kết quả của bài cũ có thể hiện ở bài mới. Cần kiểm tra ngữ cảnh và phiên bản trước khi cập nhật.

Mười ý phải nhớ trước giờ báo cáo

1. Client–server; server Java, database SQLite. 2. Có web và Swing. 3. Swing dùng TCP 5000, 5001, 5002. 4. Web dùng HTTP/HTTPS, media WebRTC. 5. Giao thức riêng nằm ở tầng ứng dụng. 6. Frame = 4 byte độ dài + UTF-8; phải đọc đủ. 7. Quyền do server quyết định. 8. Khóa, token và version có vai trò khác nhau. 9. Commit trước, cập nhật RAM/báo sau. 10. Nộp snapshot, chấm Docker, lỗi hệ thống khác bài sai.

Khi chưa nhớ chi tiết

Hãy trả lời theo bốn bước: chức năng làm gì → dữ liệu đi qua đâu → server kiểm tra gì → lưu và trả kết quả thế nào. Nếu bị hỏi đúng một hằng số chưa nhớ, nói phần nguyên lý rồi mở file để đối chiếu. Không đoán chức năng, kết quả kiểm thử hoặc số lượng người dùng đã chịu tải.

23 Tra cứu mã nguồn và thuật ngữ

Các đường dẫn dưới đây tính từ thư mục StudyFix. Đây là bản đồ để mở nhanh khi thầy yêu cầu chỉ ra phần triển khai.

Nội dung	File hoặc vị trí
Khởi động toàn bộ dịch vụ	server/src/studyfix/assignment/AssignmentEnabledServerMain.java
Lắng nghe TCP và xử lý lệnh	server/src/studyfix/Server.java; ClientHandler.java
Frame điều khiển ở hai phía	server/src/studyfix/Protocol.java; client/.../net/FrameCodec.java
Kết nối Swing	client/src/main/java/vn/studyfix/client/net/TcpControlConnection.java
Lượt sửa và nháp	server/src/studyfix/Room.java; EditSession.java
Master nhánh và transaction	server/src/studyfix/ClassroomService.java; ClassroomRepository.java
Lưu lõi và băm mật khẩu	server/src/studyfix/Store.java; Passwords.java
Bài tập và chấm code	server/src/studyfix/assignment/service/AssignmentService.java; grading/DockerJavaSandbox.java
Gateway và media	server/src/studyfix/WebGatewayServer.java; MeetingCalls.java; screenshare/
HTTP web và cuộc gọi	web-client/static/core/api.js; features/classroom/call.js
Cấu hình luồng chạy và giới hạn	README.md mục 3 đến 12

Thuật ngữ ngắn gọn

Payload: nội dung chính của bản tin. Frame: một đơn vị bản tin theo quy tắc ứng dụng. Serialization: biến dữ liệu thành dạng có thể truyền/lưu. UTF-8: mã hóa văn bản thành byte. API: điểm giao tiếp có quy tắc. Gateway: cầu nối giữa giao diện web và dịch vụ. Repository: lớp phụ trách truy cập dữ liệu. Transaction: nhóm thao tác cùng thành công hoặc hủy. Queue: hàng đợi công việc. Worker: luồng lấy công việc ra xử lý. Verdict: kết luận chấm bài. Idempotency: gửi lại cùng thao tác mà tránh tạo kết quả nghiệp vụ trùng.

Cách tự kiểm tra đã hiểu

Đóng tài liệu rồi vẽ lại sơ đồ trang 2. Tự kể luồng /login, luồng lưu nhánh và luồng nộp bài trong mỗi luồng khoảng một phút. Sau đó mở ba file tương ứng và chỉ đúng chỗ tạo socket, kiểm tra version, gọi sandbox. Nếu làm được, bạn đã có nền để trả lời phần lớn câu hỏi trọng tâm.